

1. Importer les librairies nécessaires

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import classification_report
```

2. Préparer les données

```
# Charger les données
df = pd.read_csv('HeartDiseaseUCI.csv')

# convertir la variable cible en valeur binaire (0, 1)
def set_num(x):
    if x == 0:
        return 0
    else:
        return 1

df["target"] = df["num"].apply(set_num)

# Prétraitement des données
X = df.drop(['target'], axis=1)
y = df['target']

# Diviser les données en jeux d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Pipeline de prétraitement pour les données numériques
numeric_features = X.select_dtypes(include=['int64',
'float64']).columns
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features)
```

```
1)
```

```
# Appliquer le prétraitement
X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)
```

3. Sélectionner et configurer les modèles

```
# Configurer les modèles individuels
model1 = LogisticRegression(random_state=42)
model2 = RandomForestClassifier(n_estimators=100, random_state=42)
model3 = SVC(probability=True, random_state=42)

# Créer le classificateur par vote
voting_clf = VotingClassifier(
    estimators=[('lr', model1), ('rf', model2), ('svc', model3)],
    voting='soft' # 'hard' pour le vote majoritaire, 'soft' pour le
    vote basé sur les probabilités
)
```

4. Entraîner le classificateur par vote

```
# Entraîner le modèle
voting_clf.fit(X_train, y_train)

# Évaluer le modèle
y_pred = voting_clf.predict(X_test)
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	29
1	1.00	1.00	1.00	32
accuracy			1.00	61
macro avg	1.00	1.00	1.00	61
weighted avg	1.00	1.00	1.00	61

5. Analyser les résultats